

Python range

[< Previous](#)[Next >](#)

Python range

The built-in `range()` function returns an immutable sequence of numbers, commonly used for looping a specific number of times.

This set of numbers has its own data type called `range`.

Note: Immutable means that it cannot be modified after it is created.

Creating ranges

The `range()` function can be called with 1, 2, or 3 arguments, using this syntax:

```
range(start, stop, step)
```

Call range() With One

Argument

If the range function is called with only one argument, the argument represents the **stop** value.

The **start** argument is optional, and if not provided, it defaults to 0.

`range(10)` returns a sequence of each number from 0 to 9. (The start argument, 0 is inclusive, and the stop argument, 10 is exclusive).

Example

Get your own Python Server

Create a range of numbers from 0 to 9:

```
x = range(10)
```

Try it Yourself »

Call range() With Two Arguments

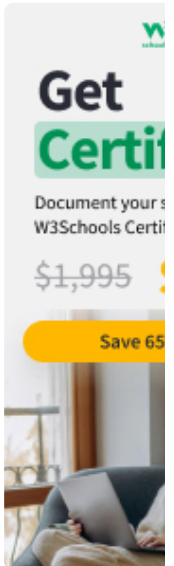
If the range function is called with two arguments, the first argument represents the **start** value, and the second argument represents the **stop** value.

`range(3, 10)` returns a sequence of each number from 3 to 9:

Example

Create a range of numbers from 3 to 9:

```
x = range(3, 10)
```



COLOR
PICKER





- Python Variables
- Python Data Types
- Python Numbers
- Python Casting
- Python Strings
- Python Booleans
- Python Operators
- Python Lists
- Python Tuples
- Python Sets
- Python Dictionaries
- Python If...Else
- Python Match
- Python While Loops
- Python For Loops
- Python Functions
- Python Decorators
- Python Range
- Python Lambda
- Python Arrays
- Python OOP
- Python Classes/Objects
- Python Inheritance
- Python Iterators
- Python Polymorphism
- Python Scope
- Python Modules
- Python Dates
- Python Math
- Python JSON
- Python RegEx
- Python PIP
- Python Try...Except
- Python String Formatting

Call range() With Three Arguments

If the range function is called with three arguments, the third argument represents the **step** value.

The step value means the difference between each number in the sequence. It is optional, and if not provided, it defaults to 1.

`range(3, 10, 2)` returns a sequence of each number from 3 to 9, with a step of 2:

Example

Create a range of numbers from 3 to 9:

```
x = range(3, 10, 2)
```

Try it Yourself »

Using ranges

Ranges are often used in **for** loops to iterate over a sequence of numbers.

Example

Iterate over each value in a range:

```
for i in range(10):  
    print(i)
```

Try it Yourself »

Using List to Display Ranges

The range object is a data type that represents an immutable sequence of numbers, and it is not directly displayable.

Therefore, ranges are often converted to lists for display.

Example

Convert different ranges to lists:

```
print(list(range(5)))  
print(list(range(1, 6)))  
print(list(range(5, 20, 3)))
```

Try it Yourself »

Slicing Ranges

Like other sequences, ranges can be sliced to extract a subsequence.

Example

Extract a subsequence from a range:

```
r = range(10)  
print(r[2])  
print(r[:3])
```

Try it Yourself »

Note: The first print statement returns the value at index 2, and the second print statement returns a new range object, from index 0 to 3.

Membership Testing

Ranges support membership testing with the `in` operator.

Example

Test if the numbers 6 and 7 are present in a range:

```
r = range(0, 10, 2)
print(6 in r)
print(7 in r)
```

Try it Yourself »

The return value is `True` when the number is present in the range, and `False` when it is not.

Length

Ranges support the `len()` function to get the number of elements in the range.

Example

Get the length of a range:

```
r = range(0, 10, 2)
print(len(r))
```

Try it Yourself »

Exercise [?]

What does range(3) produce when converted to a list?

- ☐ [1, 2, 3]
- ☐ [0, 1, 2]
- ☐ [0, 1, 2, 3]

Submit Answer »

◀ Previous

Sign in to track progress

Next ▶

ADVERTISEMENT

ADVERTISEMENT